Micro processors Systems

Extended Assembly Language Operations

Dr. Hesham El Zouka

Arithmetic Instructions – SUB, SBB, DEC, AAS, DAS, NEG

Mnemonic	Meaning	Format	Operation	Flags Affected
SUB	Subtract	SUB D, S	(D) - (S) ➔ (D) Borrow ➔ (CF)	All
SBB	Subtract with borrow	SBB D, S	(D) - (S) - (CF) ➔ (D)	All
DEC	Decrement by one	DEC D	(D) - 1 ➔ (D)	All but CF
NEG	Negate	NEG D		All CF=1
DAS	Decimal adjust for subtraction	DAS	(convert the result in AL to packed decimal format)	All
AAS	ASCII adjust for subtraction	AAS	(AL) difference (AH) dec by 1 if borrow	CY, AC

Examples: DAS

1. **AAS (ASCII Adjust AL After Subtraction)**

   The AAS instruction adjusts the AL register after a subtraction operation with BCD (Binary- Coded Decimal) values. This ensures a valid BCD result in AL after subtraction.

```
MOV AX, 38H
SUB AL,39H ;  AX=00FF
AAS        ; AX=FF09 ten's complement of –1
OR AL,30H  ; AL = 39
```

Borrow 1 from AH

2. **DAS (Decimal Adjust AL After Subtraction)**

   Similar to AAS, DAS adjusts AL for valid BCD results after subtraction. If AL's lower nibble (4 bits) exceeds 9, DAS subtracts 6 from AL and sets CF (Carry Flag).

```
MOV BL, 28H
MOV AL, 83H
SUB AL,BL;   AL=5BH
DAS       ;  adjusted as AL=55H
```

NEG (Negate)

3. **NEG (Negate)**

   The NEG instruction inverts the value of a register or memory operand by calculating its two's complement.

   Example: If BX holds `003AH`, `NEG BX` changes BX to `FFC6H`, with CF set to 1 due to overflow.

Example

- Assume (BX)=003AH, what is the result of executing the instruction

  NEG BX

  (BX)=0000000000111010

  2's comp = 1111111111000110 = FFC6H

  (CF)=1

Example

4. **Example: 32-bit Subtraction**

   Perform subtraction of two 32-bit numbers, `X` and `Y`, stored in memory starting from `200h` and `100h`, respectively:

```assembly
MOV SI, offset 200h   ; Start address for X
MOV DI, offset 100h   ; Start address for Y
MOV AX, [SI]          ; Load lower word of X
SUB AX, [DI]          ; Subtract lower word of Y
MOV [SI], AX          ; Store result in memory
```

Multiplication and Division

Mnemonic	Meaning	Format	Operation	Flags Affected
MUL	Multiply (unsigned)	MUL S	(AL) · (S8) → (AX) (AX) · (S16) → (DX),(AX)	OF, CF SF, ZF, AF, PF undefined
DIV	Division (unsigned)	DIV S	(1) Q((AX)/(S8)) → (AL) R((AX)/(S8)) → (AH) (2) Q((DX,AX)/(S16)) → (AX) R((DX,AX)/(S16)) → (DX) If Q is FF_{16} in case (1) or $FFFF_{16}$ in case (2), then type 0 interrupt occurs	OF, SF, ZF, AF, PF, CF undefined
IMUL	Integer multiply (signed)	IMUL S	(AL) · (S8) → (AX) (AX) · (S16) → (DX),(AX)	OF, CF SF, ZF, AF, PF undefined
IDIV	Integer divide (signed)	IDIV S	(1) Q((AX)/(S8)) → (AL) R((AX)/(S8)) → (AH) (2) Q((DX,AX)/(S16)) → (AX) R((DX,AX)/(S16)) → (DX) If Q is positive and exceeds $7FFF_{16}$ or if Q is negative and becomes less than 8001_{16}, then type 0 interupt occurs	OF, SF, ZF, AF, PF, CF undefined
AAM	Adjust AL for multiplication	AAM	Q((AL)/10) → (AH) R((AL)/10) → (AL)	SF, ZF, PF OF, AF,CF undefined
AAD	Adjust AX for division	AAD	(AH) · 10 + (AL) → (AL) 00 → (AH)	SF, ZF, PF OF, AF, CF undefined
CBW	Convert byte to word	CBW	(MSB of AL) → (All bits of AH)	None
CWD	Convert word to double word	CWD	(MSB of AX) → (All bits of DX)	None

Multiplication and Division

Multiplication (MUL or IMUL)	Multiplicant	Operand (Multiplier)	Result
Byte * Byte	AL	Register or memory	AX
Word * Word	AX	Register or memory	DX :AX
Dword * Dword	EAX	Register or Memory	EDX :EAX

Division (DIV or IDIV)	Dividend	Operand (Divisor)	Quotient : Remainder
Word / Byte	AX	Register or memory	AL : AH
Dword / Word	DX:AX	Register or memory	AX : DX
Qword / Dword	EDX: EAX	Register or Memory	EAX : EDX

Multiplication and Division

1. **MUL (Unsigned Multiplication)**

   Multiplies the specified operand by the contents of AL, AX, or EAX based on operand size, and stores results in AX, DX

   , or EDX

   , respectively.

2. **DIV (Unsigned Division)**

   Divides AX, DX

   , or EDX

   by a register or memory operand, placing the quotient in AL/AX/EAX and the remainder in AH/DX/EDX.

3. **IMUL and IDIV (Signed Multiplication/Division)**

   Handles signed integers, adjusting for positive or negative values in multiplication or division results.

Multiplication and Division Examples

Ex.1: **A**ssume that each instruction begins with the following values:

AL = 85H, BL = 35H, AH = 0H

1. MUL BL → *AL* x *BL* = *85H* x *35H* = *1B89H*
   Result: AX = 1B89H

2. IMUL BL → 2's complement of *AL* x *BL* = 2's comp. (85*H*) x 35*H*

   Calculation: *7BH* x *35H* = *1977H*

   Convert to 2's complement: E689H
   Result: AX = 1B89H

3. DIV BL → $\frac{AX}{BL} = \frac{0085H}{35H} = 02$ (AL), (85-02*35=1B) → AH

IDIV BL → $\frac{AX}{BL} = \frac{0085H}{35H} =$

AH	AL
1B	02

DIV BL

$AX \div BL = 0085H \div 35H$

Quotient: **AL = 02** (since $85 - 02 \times 35 = 1B$)

Remainder: **AH = 1B**

Multiplication and Division Examples (Cont.)

Ex.2: Assume that each instruction begins with the following values:

AL = F3H, BL = 91H, AH = 00H

- MUL BL → *AL* x *BL* = F3H x 91H = 89A3H → AX = 89A3H
- IMUL BL → *AL* x *BL* = 2'S AL * 2'S BL = 2'S (F3H) * 2'S(91H) = 0DH * 6FH = 05A3H → AX.
- IDIV BL → $\frac{AX}{BL} = \frac{00F3H}{2'S(91H)} = \frac{00F3H}{6FH} = 2 \rightarrow$ (00F3 – 2*6F=15H)

AH	AL
15	02
R	Q

$\rightarrow \frac{POS}{NEG} = NEG$ → 2's(02) = FEH→

AH	AL
15	FE

- DIV BL → $\frac{AX}{BL} = \frac{00F3H}{91H}$ = 01→(F3-1*91=62) →

AH	AL
62	01
R	Q

Multiplication and Division Examples (Cont.)

Ex.3: Assume that each instruction begins with the following values:
AX = F000H, BX = 9015H, DX = 0000H

1. MUL BX = F000H * 9015H =

DX	AX
8713	B000

2. IMUL BX = 2'S(F000H) x 2'S(9015H) = 1000 x 6FEB =

DX	AX
06FE	B000

3. DIV BL = $\frac{F000H}{15H}$ = B6DH → More than FFH → Divide Error.

4. IDIV BL = $\frac{2'S(F000H)}{15H} = \frac{1000H}{15H}$ = C3H > 7F → Divide Error.

Multiplication and Division Examples (Cont.)

Ex.4: Assume that each instruction begins with the following values:
AX = 1250H, BL = 90H

1. IDIV BL → $\frac{AX}{BL} = \frac{1250H}{90H} = \frac{POS}{NEG} = \frac{POS}{2'sNEG} = \frac{1250H}{2's(90H)} = \frac{1250H}{70H}$

= 29H (Q) → (1250 – 29 * 70) = 60H (REM)

29H (*POS*) → 2'S (29H) = D7H →

R	Q
60H	D7H

2. DIV BL → $\frac{AX}{BL} = \frac{1250H}{90H}$ = 20H→1250-20*90

=50H →

R	Q
50H	20H
AH	AL

Logical Instructions

Mnemonic	Meaning	Format	Operation	Affected Flags
AND	Logical AND	AND D, S	$(S) \cdot (D) \rightarrow (D)$	OF, SF, ZF, PF, CF AF undefined
OR	Logical Inclusive OR	OR D, S	$(S) + (D) \rightarrow (D)$	OF, SF, ZF, PF, CF AF undefined
XOR	Logical Exclusive OR	XOR D, S	$(S) \oplus (D) \rightarrow (D)$	OF, SF, ZF, PF, CF AF undefined
NOT	LOGICAL NOT	NOT D	$\overline{(D)} \rightarrow (D)$	None

Destination	Source
Register	Register
Register	Memory
Memory	Register
Register	Immediate
Memory	Immediate
Accumulator	Immediate

Destination
Register
Memory

Logical Instructions

- AND
  - The AND instruction can use various addressing modes, except memory-to-memory and segment registers.
  - Commonly used for clearing specific bits within a register or memory location, a process known as *masking*.
  - xxxx xxxx **AND** 0000 1111 = 0000 xxxx (clear the first four bits)
  - Examples:
    - `AND BL, 0FH` ; Clears the upper four bits of BL, leaving the lower four bits
    - `AND AL, [345H]` ; Applies the bitwise AND operation between AL and the value at memory location 345H
- OR
  - Used for setting specific bits within a register or memory location.

    xxxx xxxx **OR** 0000 1111 = xxxx 111 (Set the upper four bits)
    - `OR AL, 0FH` ; Sets the lower four bits of AL to 1 without altering the upper four bits

Logical Instructions (Cont.)

- XOR
  - Used to invert (toggle) specific bits within a register or memory location.

    xxxx xxxx XOR 0000 1111 = xxxx x'x'x'x'

  - **Example:** To clear bits 0 and 1, set bits 6 and 7, and invert bit 5 of register CL:

```
AND CL, FCH      ;      1111 1100B
OR CL, 0C0H      ;      1100 0000B
XOR CL, 020H     ;      0010 0000B
```

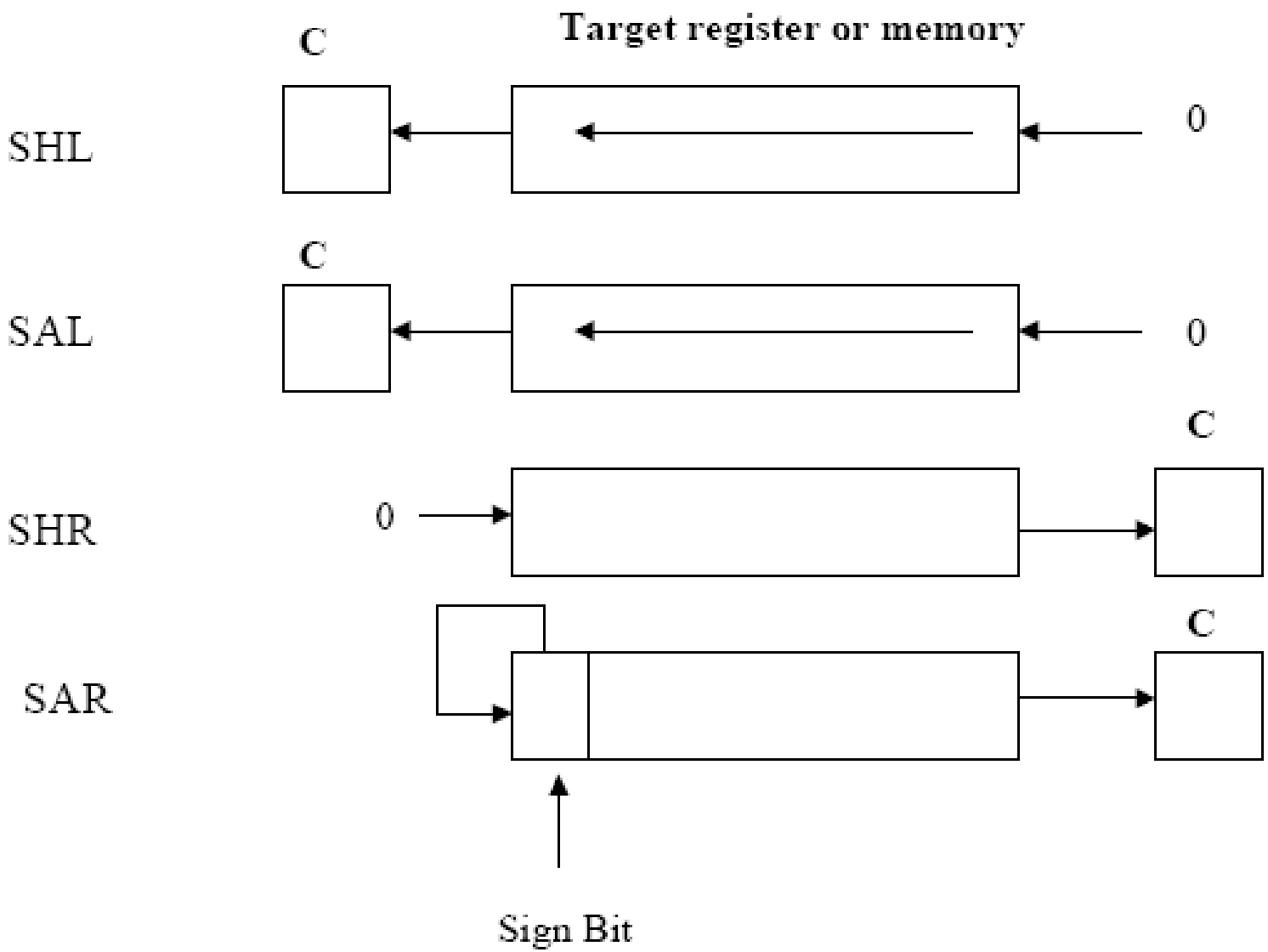
In this example:

- The `AND` operation clears bits 0 and 1 of `CL`.
- The `OR` operation sets bits 6 and 7 of `CL`.
- The `XOR` operation toggles (inverts) bit 5 of `CL`.

Shift Instructions

Mnemonic	Meaning	Format	Operation	Affected Flags
SAL/SHL	Shift arithmetic Left / shift Logical left	SAL/SHL D, Count	Shift the value in (D) to the left by the number of bit positions equal specified by Count, fills with zeros on the right.	CF, PF, SF, ZF AF: Undefined OF: undefined if Count $\neq 1$
SHR	Shift logical right	SHR D, Count	Shifts the value in (D) to the right by the number of bit positions specified by Count, filling the vacated positions on the left with zeros.	CF, PF, SF, ZF AF: Undefined OF: undefined if Count $\neq 1$
SAR	Shift arithmetic right	SAR D, Count	Shift (D) to the right by the number of bit positions specified by Count, and fill the vacated positions on the left with the original most significant bit.	CF, PF, SF, ZF AF: Undefined OF: undefined if Count $\neq 1$

Shift Instructions (Cont.)



Examples:

- Ex.1.

  Multiply the value in AX by 10.

```assembly
SHL AX, 1      ; Shift AX left by 1 (equivalent to multiplying by 2)
MOV BX, AX     ; Copy the result to BX
MOV CL, 2      ; Set CL to 2
SHL AX, CL     ; Shift AX left by 2 (equivalent to multiplying by 4)
ADD AX, BX     ; Add BX to AX (AX * 2 + AX * 4 = AX * 10)
```

- Ex.2.

  Determine the result of SAR CL, 1 when CL initially contains B6H.

  - Result: DBH

- Ex.3.

  Determine the result of SHL AL, CL when AL contains 75H and CL contains 3.

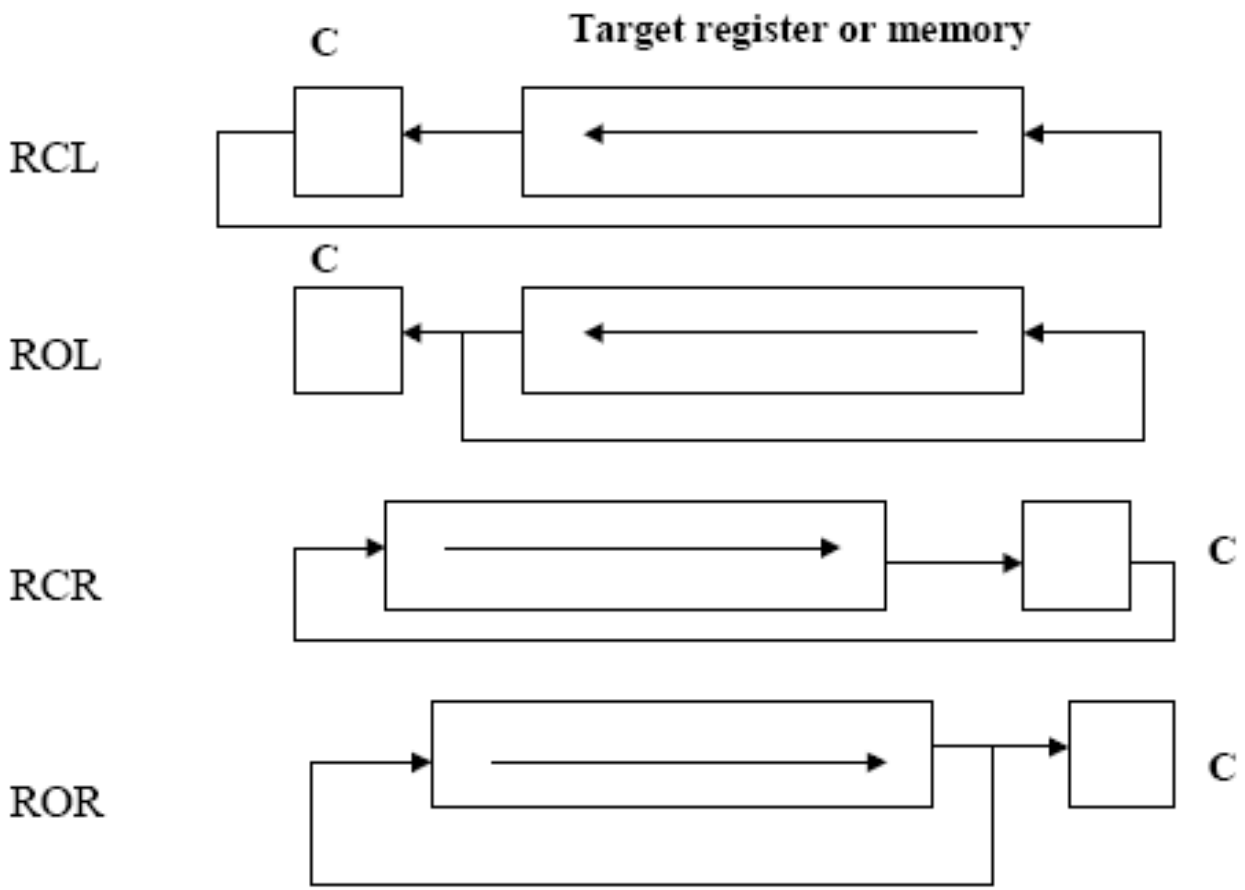
  - Result: A8H

2022 Dr. Hesham EL Zouka

Rotate Instructions

Mnemonic	Meaning	Format	Operation	Affected Flags
ROL	Rotate Left	ROL D, Count	Rotates (D) to the left by the number of bit positions specified by Count. Each bit shifted out from the leftmost position wraps around to the rightmost position.	CF OF: undefined if count $\neq 1$
ROR	Rotate Right	ROR D, Count	Rotate (D) to the right by the number of bit positions specified by Count. Each bit shifted out from the rightmost position wraps around to the leftmost position.	CF OF: undefined if count $\neq 1$
RCL	Rotate Left Through Carry	RCL D, Count	Similar to ROL, but the carry flag is included in the rotation with (D).	CF OF undefined if count $\neq 1$
RCR	Rotate Right Through Carry	RCR D, Count	Similar to ROR, but the carry flag (CF) is included in the rotation with (D).	CF OF: undefined if count $\neq 1$

Rotate Instructions (Cont.)



Ex.

What is the result of the instruction ROL BTRE PTR [SI], 1 if SI points to a memory location containing 41H?

Answer: 82H

Example

Write a program that counts the number of ones (1 bits) in a byte and stores the result in the BL register.

```
DATA1 DB 97; 61h
SUB BL, BL;   Clear BL to keep the count of number of 1's
MOV DL, 8;    Set counter to 8 (number of bits in a byte)
MOV AL, DATA1; Load DATA1 into AL
AGAIN: ROL AL,1; rotate AL left by 1 bit
JNC NEXT;    Jump to NEXT if carry Flag (CF) is not set
INC BL; if CF=1, increment the count in BL
NEXT: DEC DL; Decrement counter
JNZ AGAIN; if DL is not zero, repeat the loop
NOP        ; No operation (optional, can be removed)
```